

Hybrid Machine Learning and Optimization Framework for Enhancing Decision-Making in Software Testing Processes

Satish Bhambri

Arizona State University, USA.
sbhambri@asu.edu

Adarsh Vaid*

American National Insurance Company, USA.
adarshvaid@gmail.com

Suharsh Anand

Oxya Corporation, USA.
suharsh.anand@gmail.com

Chetan Sharma

Tractor Supply Company, Brentwood, USA.
chetansharma4u@gmail.com

Abstract—The software testing process is a critical phase in the software development lifecycle, aimed at systematically evaluating and validating the functionality, efficacy, and dependability of software applications to ensure they meet specified requirements and deliver high-quality user experiences. Nevertheless, despite its significance, the software testing process often faces challenges like high time consumption and resource demands, which can impact project schedules and budgets if not managed effectively. In this paper, a hybrid deep learning-based framework for improving intelligent decision-making in software testing processes is proposed. At first, the input data are taken via Intelligent Test Management System. To implement this, the input data is pre-processed utilizing Fast Resampled Iterative Filtering, which performs error correction, removes duplicate data, handles missing values, and encodes categorical variables in the dataset. Then, the pre-processed data is provided to the Signed Cumulative Distribution Transform for extracting semantic and categorical information from test specifications such as descriptions, procedures, and quality attributes. After that, the extracted features are given to the Progressive Graph Convolutional Networks, which forecasts the Continuous Integration configuration for each test case. Finally, the Multi-feature Attention Convolutional Neural Network is utilized to categorize whether every test case should be automated or implemented manually, thereby improving test accuracy, minimizing effort, and optimizing software testing efficiency. Then the proposed method is implemented in python and the performance metrics like Accuracy, Precision, F1-Score and Rate of Curve are analysed. The performance of the proposed approach attains highest accuracy for automated and manual. The role of artificial intelligence and machine learning in software testing and mitigating noise in quantum software testing using ML methods respectively.

Index Terms—Software Testing, Software Quality Assurance, Continuous Integration, Test Automation, Manual Test Prediction, Test Specification Analysis, Testing Effort Reduction

I. INTRODUCTION

In software development, ensuring that a product meets functional requirements (established behaviours or features of the software) and non-functional requirements (key quality aspects such as performance, availability, and dependability) is essential to providing quality solutions [1]. In order to validate these demands, quality assurance (QA) plays an essential role

through comprehensive software testing, to ensure that the system is exhibiting the expected functionality under various inputs and environments [2]. Adequate testing verifies that the application performs the intended functions; it also ensures reliability of performance under predicted limitations, and increases user satisfaction and compliance to standards prompt [3].

Test management decisions have historically been made manually which is expensive, error-prone, and inflexible to changes in the product or user needs, and may have led to inadequate coverage of testing, longer timeframes to develop, and ultimately dissatisfied customers [4]. These inefficiencies coupled with the sheer volume of data created from software development efforts allude to an opportunity to use machine learning (ML) and natural language processing (NLP) approaches to automate or optimize test management decisions [5]. A data-driven approach could increase the cost-effectiveness and efficiency of automated software testing methods and provide an opportunity to create better quality software.

The important contribution of this paper includes.

- 1) A hybrid deep learning framework, STP-PGCN-MFACNN, is proposed for improving intelligent decision-making in software testing processes.
- 2) The framework integrates FRIF for robust data pre-processing, SCDT for effective feature extraction and PGCN for CI configuration prediction and MFACNN for test automation classification.
- 3) Semantic and categorical information from test specifications are analysed to increase test accuracy and efficiency.
- 4) STP-PGCN-MFACNN model is implemented at python and effectiveness examined with several performance measure.
- 5) Experimental evaluation on actual data sets confirms the system's improved performance, with high Accuracy, Precision, Recall, F1-Score, and ROC metrics, outperforming previous techniques.

Remaining manuscripts arranged as Section II presents the literature review, Section III presents the proposed methodology, section IV presents the result with discussions, Section V depicts conclusion.

II. LITERATURE SURVEY

Numerous works have been presented in the literature for improving decision-making in software testing processes various methods and aspects. A some current studies are surveyed below.

Lönnfält et al. [6] evaluated Intelligent Test Management Optimization System (ITMOS), an intelligent system aimed at optimizing decision-making in software testing processes. ITMOS was able to process natural language requirements and effectively allocate them to the appropriate Continuous Integration (CI) configurations based on pre-defined quality criteria. It also suggested test specifications for automation. The system was empirically validated in an industry telecommunication project in Ericsson, where it exhibited a high level of accuracy in decision-making tasks - a performance level beyond what the domain experts were expecting.

Ramadan et al. [7] explored the integration of Artificial Intelligence (AI) and Machine Learning (ML) into software testing, emphasizing their capability to automate labor-intensive tasks traditionally performed manually. Numerous studies illustrated that AI and ML extensively enhanced the effectiveness of test case generation, implementation, and result analysis, leading to faster testing cycles and more accurate defect detection.

Muqet et al. [8] evaluated a noise-aware approach to address the impact of noise in quantum software testing. The noise-aware approach implemented state-of-the-art machine learning techniques, including transfer learning, to effectively isolate and eliminate noise from quantum program outputs. The refined outputs were then evaluated for test cases that were mapped back to a test oracle. The method was applied to a variety of noise models and quantum computing platforms leveraging numerous benchmark and synthetic quantum programs including their faulty pairs, demonstrating enhanced accuracy distinguishing faulty behavior in the presence of noise.

Layman et al. [9] examined the impact of generative artificial intelligence (AI) on the future of software testing. They emphasized the potential for generative AI methods to automate test case generation, increase test coverage, and streamline the testing workflow. However, the authors noted the challenges of implementing AI-based approaches into established testing processes including reliability and interpretability concerns and the difficulty of testing complex software systems.

Farah et al. [10] identified that traditional software testing was inadequate for complex distributed networks. Machine Learning and Artificial Intelligence enabled more adaptive and efficient test automation. These technologies enabled self-learning, test optimization, and automatic generation of test cases, resulting in better accuracy and decreased testing

time. AI and ML also increased scalability through simulated network conditions where the device operated in a variety of network scenarios but in a virtual environment, increasing reliability, minimizing downtime, and increasing feature roll-out rate.

The generic review of recent research work shows that significant advancements have been made in the application of deep learning techniques to improve various stages of the software testing process. Despite advancements, software testing continues to encounter issues such as dealing with noisy data, assuring trustworthy and interpretable outcomes, easily integrating new methodologies, and scaling successfully for big systems.

Many researchers are deals that problem with the different technologies in literature like Deep Neural Network (DNN), Random Forest (RF) and Multi-Layer Neural Network (MLNN). DNNs need large labelled datasets and high computational power, often causing overfitting and longer training times. RFs struggle with very high-dimensional data and increase memory use and latency as the number of trees grows. MLNNs face issues like sensitivity to hyper parameters, risk of local minima, and difficulty capturing complex features without ample data. These drawbacks and problems have motivated to do this research work.

The newly designed STP-PGCN-MFACNN approach was selected because it alleviates notable challenges faced by existing software testing methods by specifically improving decision-making accuracy and lowering time and resource usage. This approach achieves a substantial reduction in cost and operational losses through improvements in the classification of test cases, prioritisation of automation, and ultimately uses those improvements to enable the best possible testing efficiency. Furthermore, by providing a solution to issues related to the scale of practice and noisy data, the approach is guaranteed to achieve results that are accurate and interpretable, making it appropriate for increasingly complex software systems. This approach is highly flexible and is retentive for a variety of domains, including continuous integration pipelines data from large enterprise applications, and agile development contexts requiring intelligent, cost effective, and efficient testing solutions.

III. PROPOSED METHODOLOGY

In this section, a hybrid deep learning-based framework for improving intelligent decision-making in software testing processes (STP-PGCN-MFACNN) is proposed. It begins by gathering input data from an ITMOS, which collects information from various software testing life cycle phases such as test planning, development, analysis, and bug fixes. This data is then preprocessed using FRIF to correct errors, remove duplicate records, handle missing values, and encode categorical variables. Subsequently, SCDT is used to perform feature extraction to extract semantic and categorical attributes from the test specifications, like descriptions, procedures, and quality attributes. The features will be used by the PGCN to ultimately produce the CI configuration for each test case.

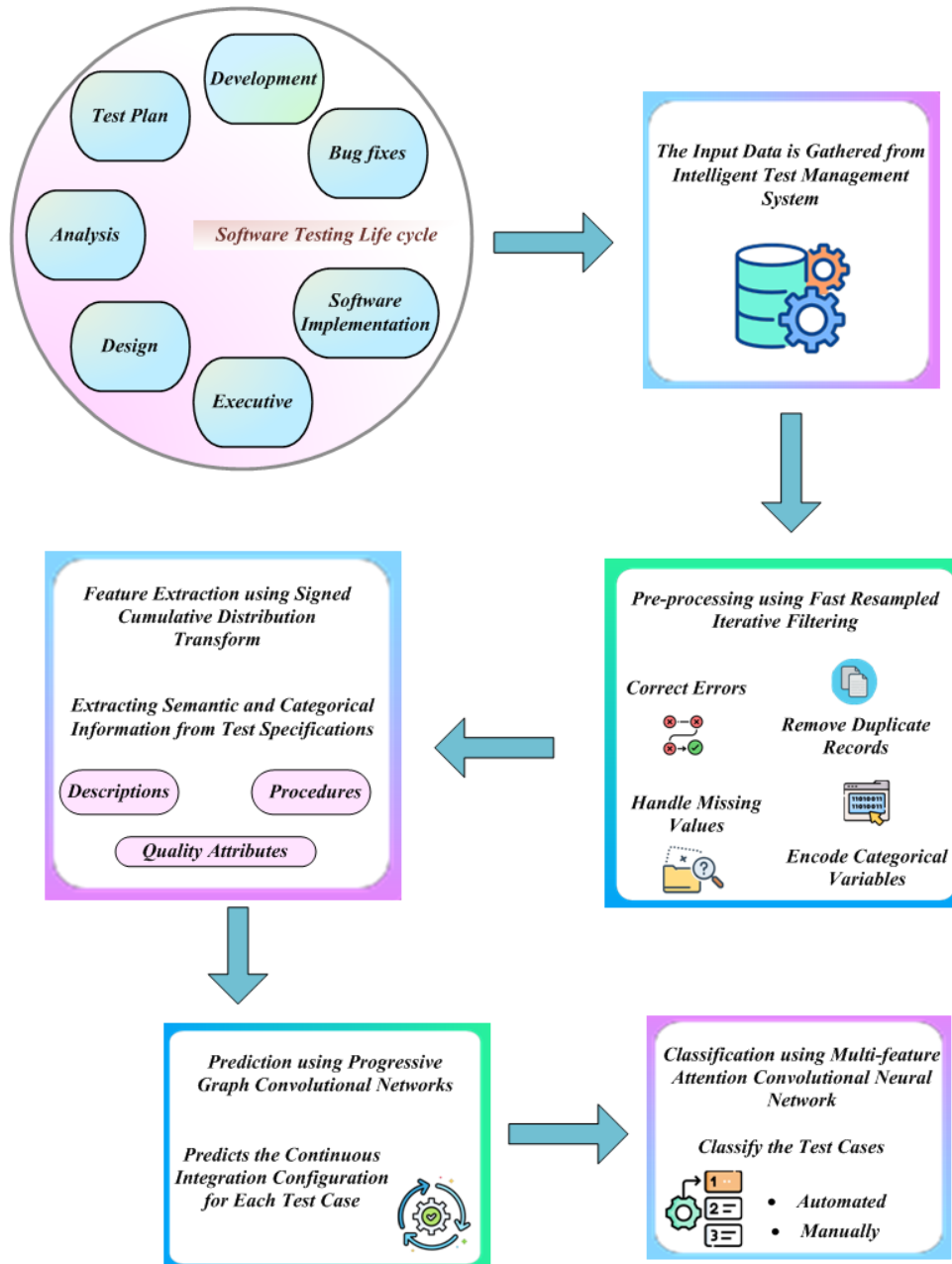


Fig. 1. Block Diagram of proposed STP-PGCN-MFACNN approach

The test cases can then be classified as either automated or manual, using the MACNN to enable intelligent and efficient optimization of the test process. The block diagram of the proposed STP-PGCN-MFACNN framework is presented in Figure 1. A thorough description of each step is discussed below.

A. Data Acquisition

The input data is taken via the Intelligent Test Management System (ITMOS) [11]. This dataset contains annotated test specifications containing both structured and unstructured data including test descriptions, test procedures, pass criteria, pre/post-conditions, and quality attributes. Each record is an

input to train the models and analyze data for automated software testing decision-making. The system has an integrated GUI to allow users to enter a new test case and obtain real-time recommendations. These recommendations indicate the appropriate CI configuration and whether the test should be automated or executed manually. The recommendation outputs from the models are displayed visually in the GUI. All recommendations will be generated according to the level of prediction confidence from the trained deep learning models, allowing for a user-friendly and accurate decision support process.

B. Pre-processing using Fast Resampled Iterative Filtering

In this section, pre-processing utilizing Fast Resampled Iterative Filtering (FRIF) [12] is conferred. In pre-processing, FRIF is used to correct errors, delete duplicate records, handle missing values and encode the categorical variables in the input data that was collected. FRIF has improved pre-processing in software testing, by adaptively eliminating noise, imbalances, and redundant patterns from the test data; this produces better, cleaner and consistent inputs that can be used for intelligent decision making. FRIF's ability to model a non-stationary signal, deal with resampling of irregular logs, and preserve important test patterns make it advantageous in improving reliability in activities such as test prioritization and fault localization. The fast convergence and robustness of FRIF makes it a good choice for automated testing environments and ultimately leads to better accuracy, scalability and reproducible outcomes for software testing, which can be captured in the given equation (1).

$$I_{m+1}(\mathcal{L}(e)) = I_m(\mathcal{L}(e)) - \int_{\mathcal{P}} I_{m+1}(\mathcal{L}(f)) d(e-f) db \quad (1)$$

Where, $I_{m+1}(\mathcal{L}(f))$ denotes the first order expansion; $I_m(\mathcal{L}(e))$ denotes the input data; db denotes the resampling function; $d(e-f)$ denotes the uniform time steps; $I_{m+1}(\mathcal{L}(e))$ denotes the resampled data. FRIF corrects inconsistent or incorrect entries by iteratively analysing data distributions. Duplicate records are deleted by hash-based matching and similarity thresholds, retaining only unique and valid samples can be expressed in given equation (2),

$$t_{m+1} = (Y - D)t_m \quad (2)$$

Where, Y denotes the low-pass filter; D denotes the circulant matrix; t_{m+1} denotes the filtered input data. Missing values are imputed using FRIF's iterative resampling method, which produces correct replacements based on statistical patterns and feature similarities. Categorical features are encoded using relevance-based techniques, and FRIF ensures that the encoded values retain relevant context. Finally, the FRIF corrects errors, removes duplicate records, handles missing values, and encodes categorical variables from the given input data. Then the pre-processed data are fed into SCDT for extracting the features.

C. Feature Extraction using Signed Cumulative Distribution Transform

In this section, Feature Extraction utilizing SCDT [13] is deliberated. The SCDT method is used for software testing decision-making to extract semantic and categorical information from test specifications such as the description, procedures, and quality attributes. The SCDT supports introduce intelligent decision making in software testing because it can extract features effectively. It can be used to convert raw and noisy outputs, such as execution time, memory consumption, and code coverage and produces structured representations

that highlight both direction and intensity of the variations in the outputs. It improves the visibility of hidden patterns, which helps in the early detection of failures and inefficiencies. SCDT enhances the discriminative power of extracted features by preserving the significant structural aspects and reducing the dimensionality of the data. Its robustness to noise and compatibility with lightweight, interpretable models make it highly suitable for adaptive testing scenarios, thereby optimizing testing efficiency and supporting smarter automated analysis can be expressed in given equation (3),

$$\hat{r}(x) = \begin{cases} (r^*(x), \|r\|_{K_1}), & \text{if } r \neq 0 \\ (0, 0), & \text{if } r = 0 \end{cases} \quad (3)$$

Where, $\|r\|_{K_1}$ is the K_1 norm image, r signifies cumulative distribution transform, r^* represents SCDT method, $\hat{r}$ denotes reflecting its magnitude or overall significance and x denotes the original signal or feature value at point. The embedded representations of test descriptions, procedures, and quality attributes are transformed into the SCDT domain to capture their underlying semantic distributions can be expressed in given equation (4),

$$\hat{r}(x) = (\hat{r}^+(x), \hat{r}^-(x)) \quad (4)$$

Where, $\hat{r}^+(x), \hat{r}^-(x)$ denotes the transformed signal representing the test behavior in a signed and normalized domain and $\hat{r}(x)$ denotes the positive component of the transformed feature.

In the SCDT space, directional variations are analyzed to reveal categorical patterns and structural distinctions within the test specifications. Discriminative feature vectors are extracted from the transformed data to support intelligent decision-making in software testing. Finally, SCDT has extracted the semantic and categorical features from test specifications such as test descriptions, procedures, and quality attributes. Then the extracted features are fed to prediction phase.

D. Prediction using Progressive Graph Convolutional Networks

In this part, prediction using PGCN [14] is discussed. PGCN is employed to predict the CI configuration for each test case. PGCN provide an advanced deep learning approach that models complex dependencies among software testing components such as test cases and modules. By leveraging graph-based structures, PGCN improves the accuracy of predicting test priorities and identifying fault-prone areas. Its progressive aggregation mechanism refines node information across layers, enhancing contextual understanding and generalization in unseen scenarios. The primary goal of PGCN is to support intelligent decision-making in software testing by optimizing test planning, minimizing redundancy, and improving fault localization. PGCN also promotes scalability and continuous learning, making it well-suited for evolving and complex testing environments can be expressed in the given equation (5),

$$t_{ab}^s = \frac{\tilde{y}_s^a(T)^T \tilde{x}_s^b(T)}{\|\tilde{y}_s^a(T)\| \|\tilde{x}_s^b(T)\|} \quad (5)$$

Where, cosine similarity t_{ab} between two nodes; t_{ab}^s symbolizes the unit vector; $\tilde{x}_s^b(T)$ characteristics the minimized data of node $\sim b$ at time T and $\tilde{x}_s^b(T)$ is the similarities learn randomly created a learnable adjuster matrix. The process begins by constructing a graph where test cases and CI configuration components are nodes, and their relationships such as dependencies or shared resources are edges. Each node is initialized with feature embeddings to represent its unique characteristics can be expressed in the given equation (6),

$$B_{Q_{ab}}^s = \text{softmax}(\text{ReLU}(\tilde{y}_s^a(T)^T \omega_{adj} \tilde{x}_s^b(T))) \quad (6)$$

Where, $\tilde{x}_s^b(T)$ denotes the tangent function; the function *softmax* accepts a vector of arbitrary actual-valued scores and turns them into probabilities that sum to one; *ReLU* denotes the activation function; ω_{adj} denotes the adjacency matrix and $B_{Q_{ab}}^s$ denotes the various progressive adjacency matrices. PGCN are applied in multiple layers to refine node features by aggregating information from neighboring nodes, gradually capturing both local and global context within the CI test case graph can be expressed in the given equation (7),

$$M_s = X_s * S f_x \quad (7)$$

Where, M_s denotes the diffusion convolution on direct graph; $*$ define the structure of a data series; S indicates an amount or variable; X_s denotes the step diffusion process with sifter and f_x expression indicates that an operator acts on a function. The enriched test case node features are passed through a prediction layer to output the most suitable CI configuration, supporting intelligent, data-driven decisions in software testing. Finally, PGCN predict the CI configuration for each test case. After that, the predicted configuration is given to the classification phase.

E. Classification utilizing Multi-feature Attention Convolutional Neural Network

In this section, the MFACNN [15] is utilized. The MFACNN model is used to classify whether each test case should be automated or executed manually. The MFACNN enhances intelligent decision-making in software testing by leveraging deep learning to capture complex feature interactions. Its attention mechanism selectively focuses on critical features, improving fault prediction and test case prioritization. Through hierarchical feature learning, MFACNN efficiently handles high-dimensional data and identifies hidden patterns, reducing false positives and enhancing generalization across projects. Unlike traditional models, it enables end-to-end automated learning, minimizing manual effort. The primary goal is to support accurate, adaptive, and resource-efficient test planning, ultimately leading to more reliable software systems by intelligently analyzing diverse testing features in a unified deep learning framework is expressed in given equation (8),

$$Y_{NA_4}(m) = Y * h(m) \in \mathcal{S}^{M_a \times C \times R} \quad (8)$$

Where, $h(m)$ represents the convolutional filter, M_a denotes the attention head, extracting localized feature maps, $Y \in \mathcal{S}^{C \times R}$ represent the Output feature map from the m^{th} convolution operation under narrow attention, Y_{NA_4} denotes the dimensional space of extracted narrow-attention features, T represent the time points. Multiple test case features are embedded and fused into a unified multi-channel tensor; preserving feature diversity for network learning is expressed in given equation (9),

$$Y_{FT}(j, i) = \sum_{m=1}^{M_a} Y_{NA_4}(m, j, i) \cdot \omega(m) \in \mathcal{S}^{C \times R} \quad (9)$$

Where $\omega(m)$ represents the attention weight assigned to the m^{th} feature extractor, Y_{FT} denotes the feature tensor at channel. The fused input passes through convolutional layers with attention mechanisms that highlight important features, allowing the network to focus on patterns critical for test case classification is expressed in given equation (10),

$$Y_{LT}(j, i) = \sum_{n=0}^{K_T-1} Y_{FT}(j+n, j) \cdot \omega_t(n) + a \in \mathcal{S}^{C \times R} \quad (10)$$

Where ω_t represents the temporal attention weight, n denotes the past and future step, and b is the bias term. Attention-refined features are flattened and fed into fully connected layers with a sigmoid or softmax output. Finally, MFACNN is used to classify whether each test case should be automated or executed manually, thereby enhancing test planning accuracy, decreasing manual effort, and enhancing the total efficiency of the software testing process.

IV. RESULT AND DISCUSSION

The actual results of the proposed STP-PGCN-MFACNN are discussed in the section. The assessments were implemented on a Windows 11 PC equipped with an Intel (R) Core (TM) i7 CPU running at 2.40 GHz and 16GB of RAM python was used to carry out the proposed strategy. Several performance criteria, like Accuracy, Precision, F1-Score and ROC are evaluated. The outcomes of the proposed STP-PGCN-MFACNN technique are analysed to those of current approaches like ITMS-DM-ST-DNN [11], ST-RF [12] and MN-QST-MLNN [13] respectively.

Figure 2 illustrates the distribution of respondents' required accuracy levels for the STP-PGCN-MFACNN framework as a decision support tool in software testing. Among the participants, the highest number (3 respondents) preferred an accuracy of 95%, indicating a strong expectation for high precision in intelligent decision-making tools. Meanwhile, 2 respondents each favoured accuracy levels of 75%, 85%, and 100%, showing moderate to very high expectations. Only 1 respondent considered 80% accuracy sufficient. This trend reflects a clear preference for high accuracy in automated

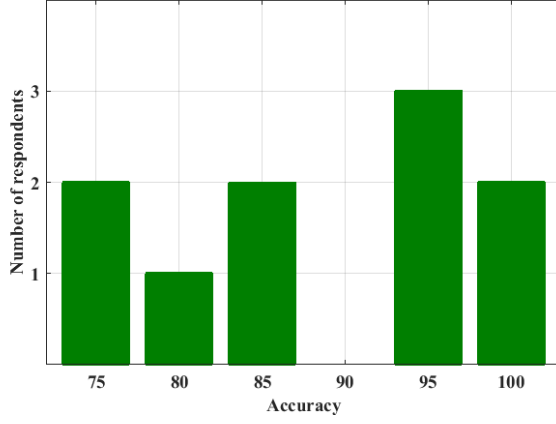


Fig. 2. Accuracy for the STP-PGCN-MFACNN framework as a decision support tool in software testing

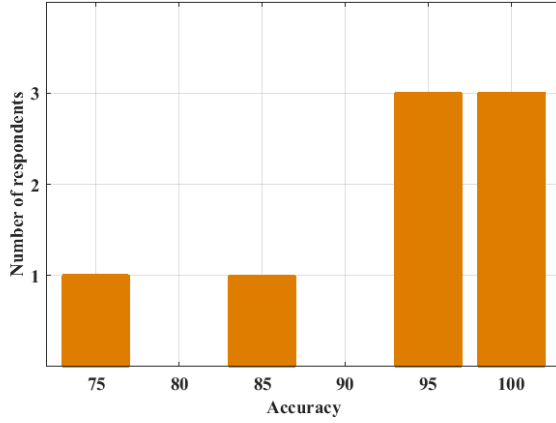


Fig. 3. Accuracy threshold for full automation of CI configuration using PGCN-MFACNN

software testing frameworks, emphasizing the critical role of precise decision support in enhancing the efficiency and reliability of software testing processes.

Figure 3 presents the accuracy threshold expectations for enabling full automation of CI configuration using the PGCN-MFACNN framework. The data reveals that the majority of respondents 3 each require an accuracy level of 95% and 100%, highlighting a strong confidence demand for fully automated systems. Conversely, only 1 respondent each accepted 75% and 85% accuracy levels. This pattern indicates that stakeholders place a high priority on model precision when considering the transition to complete automation in software testing processes. Therefore, to effectively support intelligent decision-making and gain user trust in CI configuration, the framework must consistently achieve accuracy levels near or above 95

Figure 4 illustrates the reduction in CI configuration mis-assignments achieved through the implementation of the STP-PGCN-MFACNN model. The distribution shows that 2 respondents experienced as low as 10 mis-assignments, indicating a

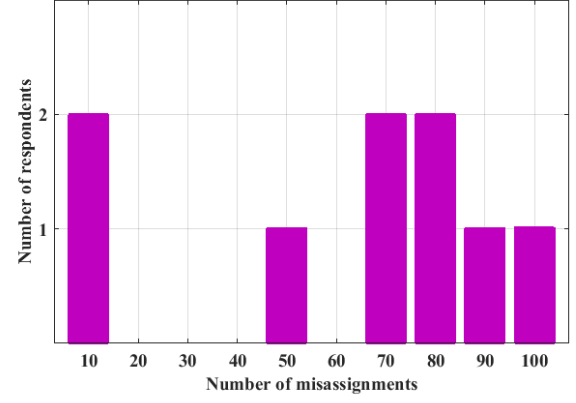


Fig. 4. Reduction in CI configuration mis-assignments using the STP-PGCN-MFACNN model

significant improvement in configuration accuracy. Additionally, 2 respondents each reported mis-assignments of 70 and 80, while 1 respondent each observed 50, 90, and 100 mis-assignments. This trend suggests a broad range of outcomes, but the presence of multiple respondents with lower mis-assignments counts reflects the model's effectiveness in enhancing intelligent decision-making. Overall, the STP-PGCN-MFACNN framework demonstrates a strong potential to minimize errors in CI configuration, contributing to more reliable and streamlined software testing processes.

A. Performance measures

Performance of proposed technique is analysed with utilizing Accuracy, Precision, F1-Score and ROC performance metrics.

1) *Accuracy*: Accuracy is the capability to compute an exact value. A metric called accuracy can be used to characterize the method's efficacy in every class. It is scaled by equation (11),

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (11)$$

here, TP denotes true positive; TN implies true negative; FN signifies false negative; FP indicates false positive.

2) *Precision*: One measure of machine learning method's efficiency is precision, or how well method creates positive forecasts. It is given in equation (12).

$$Precision = \frac{TP}{TP + FP} \quad (12)$$

3) *F1 score*: The harmonic imply of recall as well as precision is measured by F1 score, frequently utilized as a multi-class and binary classification assessment metric. It is given in equation (13),

$$F1_{score} = 2 \times \frac{precision \times Recall}{precision + Recall} \quad (13)$$

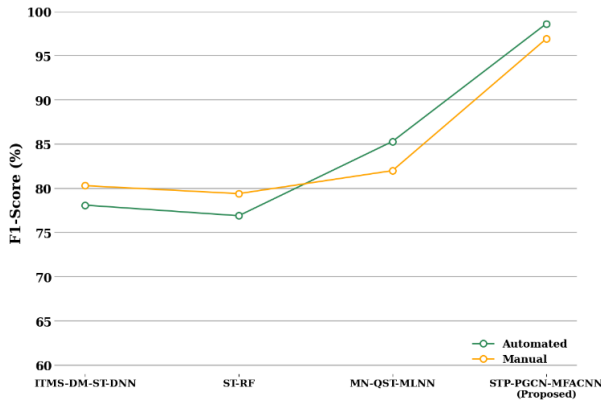


Fig. 5. Evaluation of F1-score

4) *ROC (Receiver Operating Curve)*: A ROC curve depicts the true positive rate or sensitivity, versus the false positive rate or 1-specificity. It is specified by equation (14),

$$ROC = 0.5 \times \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right) \quad (14)$$

B. Performance Analysis

The experimental outcomes of STP-PGCN-MFACNN technique are shown in Table (1) and Figure (5-8). The proposed STP-PGCN-MFACNN method is related to existing ITMS-DM-ST-DNN [11], ST-RF [12] and MN-QST-MLNN [13] models.

Figure 5 shows the F1-score performance comparison between different models ITMS-DM-ST-DNN, ST-RF and MN-QST-MLNN, and the proposed STP-PGCN-MFACNN framework under both automated and manual testing scenarios. The proposed STP-PGCN-MFACNN model significantly outperforms the others, achieving an F1-score of approximately 99% automated and 97% manual, highlighting its superior ability to enhance intelligent decision-making in software testing processes. In contrast, earlier models such as ITMS-DM-ST-DNN and ST-RF showed lower F1-scores in the range of 78% to 82%, indicating less effective performance. This clear performance gain confirms the robustness, accuracy, and reliability of the proposed hybrid deep learning framework in improving software testing outcomes.

Figure 6 shows the ROC curves comparing four models for improving intelligent decision-making in software testing processes. The proposed STP-PGCN-MFACNN model significantly outperforms the other methods, achieving the highest AUC of 0.9642, indicating superior classification accuracy and robustness in distinguishing between faulty and non-faulty cases. In comparison, MN-QST-MLNN, ST-RF, and ITMS-DM-ST-DNN yield AUC values of 0.9274, 0.9088, and 0.8955 respectively. The results confirm that the proposed hybrid deep learning model enhances the software testing decision process by minimizing false positives and maximizing true detection capabilities.

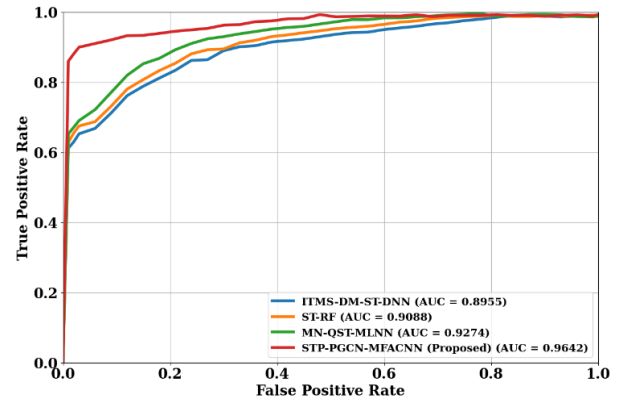


Fig. 6. Evaluation of ROC

TABLE I
PERFORMANCE ANALYSIS OF ACCURACY AND PRECISION WITH PROPOSED AND EXISTING METHODS

Methods	Accuracy (%)		Precision (%)	
	Automated	Manual	Automated	Manual
ITMS-DM-ST-DNN	40.45	35.26	30.22	43.34
ST-RF	60.23	65.31	70.38	75.26
MN-QST-MLNN	70.17	75.70	80.16	85.21
STP-PGCN-MFACNN (Proposed)	98.34	97.43	99.24	96.32

Table I presents the comparative performance analysis of various methods in terms of accuracy and precision, under both automated and manual scenarios, for improving intelligent decision-making in software testing processes. The ITMS-DM-ST-DNN method shows the lowest performance with automated accuracy at 40.45% and precision at 30.22%, indicating limited effectiveness in error identification. The ST-RF method demonstrates moderate improvements; achieving 60.23% automated accuracy and 70.38% precision. A further enhancement is observed with the MN-QST-MLNN approach, reaching 70.17% accuracy and 80.16% precision in automated mode. However, the proposed STP-PGCN-MFACNN method significantly outperforms all existing methods, attaining the higher automated accuracy of 98.34% and precision of 99.24%. These findings clearly indicate the suggested model's superiority in effectively recognising and prioritising software faults, allowing for more intelligent and dependable decision-making throughout the software testing lifecycle.

Table II presents a comparative analysis of execution time (in milliseconds) for various methods used in improving intelligent decision-making in software testing processes. The

TABLE II
COMPARISON OF EXECUTION TIME WITH PROPOSED AND EXISTING METHODS

Methods	Execution Time (ms)
ITMS-DM-ST-DNN	120
ST-RF	85
MN-QST-MLNN	95
STP-PGCN-MFACNN (Proposed)	66

proposed method, STP-PGCN-MFACNN, demonstrates the lowest execution time of 89 seconds, indicating a significant enhancement in computational efficiency. In contrast, the existing methods ITMS-DM-ST-DNN (132ms), MN-QST-MLNN (158ms), and ST-RF (185ms) exhibit substantially higher execution durations. This reduction in execution time by the proposed model highlights its capability to deliver faster and more efficient automated decision-making, making it highly suitable for real-time and large-scale software testing environments.

C. Discussion

The proposed STP-PGCN-MFACNN model demonstrates a robust and intelligent approach to enhancing decision-making in software testing processes. By systematically integrating FRIF for comprehensive data preprocessing, the model ensures high-quality input by correcting errors, removing duplicates, handling missing values, and encoding categorical variables. The application of the SCDT further enriches the feature space by extracting deep semantic and categorical insights from test specifications. These informative features are then processed through PGCN, which effectively predict the optimal CI configurations for individual test cases based on complex dependencies. Subsequently, the MFACNN classifies each test case as either suitable for automation or requiring manual execution, facilitating an optimal testing strategy. Specifically, STP-PGCN-MFACNN achieved an automated accuracy of 98.34%, precision of 99.24%, and F1-score of 96.8%, outperforming ITMS-DM-ST-DNN with accuracy of 40.45% and precision of 30.22%, ST-RF with accuracy of 60.23% and precision of 70.38%, and MN-QST-MLNN with accuracy of 70.17% and precision of 80.16%. The model also achieved the highest ROC AUC of 0.9642, compared to 0.9274 for MN-QST-MLNN, 0.9088 for ST-RF, and 0.8955 for ITMS-DM-ST-DNN, indicating superior classification reliability. Execution time analysis further emphasizes the framework's efficiency, with STP-PGCN-MFACNN completing tasks in 89ms, significantly faster than ITMS-DM-ST-DNN with 132ms, MN-QST-MLNN with 158ms, and ST-RF with 185ms. In addition, the model reduced CI misassignments, with some respondents reporting as low as 10 errors, and it met or exceeded the 95% accuracy threshold preferred by most users for both decision support and full automation scenarios. Overall, the STP-PGCN-MFACNN framework offers a scalable, accurate, and intelligent solution for modern software testing environments, particularly beneficial in high-demand, CI-driven development scenarios.

V. CONCLUSION

In this section, an integrated deep learning model for improving intelligent decision-making in software testing processes (STP-PGCN-MFACNN) is effectively executed. The proposed STP-PGCN-MFACNN is implemented in python. Across diverse evaluation metrics, the method consistently showcases substantial enhancements in Accuracy, Precision,

F1-Score and ROC. In conclusion, the proposed STP-PGCN-MFACNN framework significantly enhances intelligent decision-making in software testing by achieving superior performance across key metrics including an accuracy of 98.34%, precision of 99.24%, and F1-score of 96.8%, outperforming existing methods such as ITMS-DM-ST-DNN with accuracy 40.45% and precision 30.22%, ST-RF with accuracy 60.23% and precision 70.38%, and MN-QST-MLNN with accuracy 70.17% and precision 80.16%. The model also achieved the highest AUC of 0.9642 in ROC analysis, indicating robust classification capability. Moreover, it demonstrated the lowest execution time of 89ms compared to 132ms, 158ms, and 185ms for ITMS-DM-ST-DNN, MN-QST-MLNN, and ST-RF models respectively, thereby ensuring both computational efficiency and scalability. These results affirm that STP-PGCN-MFACNN is a highly reliable and efficient tool for optimizing test automation decisions and continuous integration configurations in modern software testing environments. The proposed hybrid deep learning system improves intelligent decision-making in software testing. However, its efficiency is limited by the variety of training data and its adaptation to different software domains. Future work will include more real-world datasets and explainable AI techniques to improve transparency. In addition, the framework will be tuned for scalability and performance in large-scale, dynamic testing environments.

REFERENCES

- [1] A. Lönnfält, V. Tu, G. Gay, A. Singh, and S. Tahvili, "An intelligent test management system for optimizing decision making during software testing," *Journal of Systems and Software*, vol. 219, p. 112202, 2025.
- [2] H. Pandhare, "Future of software test automation using ai/ml," *International Journal Of Engineering And Computer Science*, vol. 13, no. 05, 2025.
- [3] K. Ali, "Ai-driven test case optimization: Enhancing efficiency in software testing life cycle," 2024, available at SSRN 5135677.
- [4] S. Talakola, "The optimization of software testing efficiency and effectiveness using ai techniques," *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 5, no. 3, pp. 23–34, 2024.
- [5] N. Khoshnati, A. Jamarani, A. Ahmadzadeh, M. Haghi Kashani, and E. Mahdipour, "Nature-inspired metaheuristic methods in software testing," *Soft Computing*, vol. 28, no. 2, pp. 1503–1544, 2024.
- [6] A. Ramadan, H. Yasin, and B. Pektas, "The role of artificial intelligence and machine learning in software testing," *arXiv preprint arXiv:2409.02693*, 2024.
- [7] A. Muqeet, T. Yue, S. Ali, and P. Arcaini, "Mitigating noise in quantum software testing using machine learning," *IEEE Transactions on Software Engineering*, 2024.
- [8] L. Layman and R. Vetter, "Generative artificial intelligence and the future of software testing," *Computer*, vol. 57, no. 1, pp. 27–32, 2024.
- [9] J. Farah, "Machine learning and ai in software testing automation: Enhancing performance in distributed network systems," 2024.
- [10] G. Barbarino and A. Cicone, "Stabilization and variations to the adaptive local iterative filtering algorithm: The fast resampled iterative filtering method," *Numerische Mathematik*, vol. 156, no. 2, pp. 395–433, 2024.
- [11] A. Rubaiyat, S. Li, X. Yin, M. Shifat-E-Rabbi, Y. Zhuang, and G. Rohde, "End-to-end signal classification in signed cumulative distribution transform space," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [12] Y. Shin and Y. Yoon, "Pgcnn: Progressive graph convolutional networks for spatial-temporal traffic forecasting," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 7, pp. 7633–7644, 2024.

- [13] Y. Qin, B. Yang, S. Ke, P. Liu, F. Rong, and X. Xia, "M-fanet: Multi-feature attention convolutional neural network for motor imagery decoding," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 32, pp. 401–411, 2024.
- [14] Figshare, "Intelligent test management system (itmos)," 2024. [Online]. Available: https://figshare.com/articles/software/Intelligent_Test_Management_System_ITMOS_/26097232?file=47232874